Python Project Specifications

Personal Finance Tracker

Phase 1 Due: August 30, 2026, 11:55 p.m. (Central)

Phase 2 Due: September 13, 2026, 11:55 p.m. (Central)

Phase 3 Due: September 27, 2026, 11:55 p.m. (Central)

Phase 4 Due: October 11, 2026, 11:55 p.m. (Central)

This is a team project. You will work together as a team to complete the project. Each team will have five or six students.

This project is designed to provide you with hands-on experience developing a comprehensive Python application, allowing you to apply the skills and concepts you learn throughout the course.

Project Overview

Develop a comprehensive Personal Finance Tracker application to help users manage their income, expenses, and savings efficiently. This application will feature transaction recording, categorization, summary generation, and visualization functionalities. Additionally, it will incorporate Turtle graphics to represent financial data creatively.

Project Requirements

- **Integrated Development Environment (IDE)**: You may use your preferred IDE for coding.
- **Modularity**: Ensure your code is modular, utilizing appropriately sized functions and components. This will enhance readability and maintainability.
- **Transaction Recording**: Implement functionality to record income and expense transactions. Each transaction should include details such as amount, date, and category.
- **Categorization**: Enable users to categorize transactions (e.g., food, utilities, entertainment).
- **Summary Generation**: Develop a feature to generate summaries of income, expenses, and savings over specified periods (e.g., weekly, monthly).

- **Visualization**: Create at least one visualization for each report using Turtle Graphics or another appropriate visualization tool.
- **External Code Modules**: You are welcome to integrate publicly available external code libraries into your project. Be sure to document these libraries in the Project Report for each phase. You are also welcome to include external code modules developed by your team to enhance functionality.

Project Phases and Outcomes

The project consists of the following four phases designed to ensure systematic development and effective collaboration among team members. Each phase builds on the previous one, progressively adding more features and complexity to the application.

- Phase 1: Project Proposal – Develop a detailed project proposal that outlines the features, design, functionality, and development timeline for the Personal Finance Tracker application.
- Phase 2: Transaction Recording – Implement the core functionality to record and categorize income and expense transactions, ensuring data is stored effectively.
- Phase 3: Reporting and Exception Handling – Add reporting features, enhance data validation, build exception handling, and create a command-line interface for user interaction.
- Phase 4: User Interface and Final Integration – Develop a user-friendly interface, create visualizations using Turtle Graphics, and integrate all functionalities into a cohesive application.

Detailed specifications for each phase are included below. This phased approach ensures a structured development process, promotes teamwork, and allows for iterative improvements.

Flexibility in Requirements

In alignment with the gameful learning philosophy, the requirements for each project phase are intentionally written at a high level to encourage teams to employ creative and innovative solutions in developing the application. This approach allows for diverse interpretations and implementations, fostering a learning environment where students can explore different methodologies and techniques.

- Choice: By not prescribing detailed steps, teams have the freedom to experiment, problem-solve, and apply unique ideas that reflect their collective strengths and individual talents. This autonomy supports your ability to tailor the project to your own vision and creativity, enhancing your engagement and ownership of the learning process.
- Feedback: As you work through the project, you will receive timely and descriptive feedback through rubrics, auto-graded assignments, and peer reviews. This feedback will help you understand your progress, areas for improvement, and guide your creative process.
- Freedom to Fail: The flexible requirements are designed to minimize the risks associated with failure, encouraging you to explore assignments outside your comfort zone. This safe space for experimentation promotes a growth mindset, allowing you to expand your skill set without fearing negative consequences.
- Building Up Points from Zero: The project grading policy allows you to start at zero and build your way up, reflecting your efforts and achievements throughout the course. This approach supports personalized learning paths and accommodates diverse student needs and situations.

You should document any assumptions you make in interpreting the requirements in the Project Report, ensuring clarity and transparency in your design choices. This flexibility aims to enhance engagement and promote a deeper understanding of the programming concepts by allowing you to tailor the project to your own vision and creativity.

Suggested Challenge Levels

To integrate the gameful learning philosophy and accommodate varying levels of expertise, each phase includes some suggested challenge levels. You can choose the level of complexity for your team's solution to the project, promoting progressive learning and offering tailored guidance and challenges.

These levels are designed to support both beginners and advanced coders, providing options that range in complexity and depth of experience. However, you are not limited to the suggested challenge levels; you are encouraged to create challenges that align with your interests and goals. Regardless of the chosen or self-designed level, everyone will be graded equitably, ensuring that the challenge level does not affect the assigned points. This approach fosters a sense of achievement and allows you to explore different

methodologies and techniques in a fun and engaging way, enhancing your learning experience without impacting your grades.

By offering these challenge levels and the flexibility to design your own, the project specifications provide an environment where you can push boundaries, experiment with new ideas, and develop skills at your own pace. This flexibility encourages creativity, innovation, and a deeper understanding of the programming concepts, making the learning process both enjoyable and rewarding.

Suggested Team Roles and Responsibilities

To ensure effective collaboration and workload distribution, teams should rotate roles with each phase. Teams should document the roles of individual team members in the Project Report documentation. These roles include:

- **Team Leader**: Coordinates the team's efforts, ensures timely completion of tasks, and integrates all parts of the project.
- **Developer(s)**: Responsible for coding specific features or functionalities.
- **Tester/Documenter**: Tests the application for bugs, ensures code meets requirements, and documents the code and processes.

You will see a suggested work breakdown structure for each project phase. Feel free to modify the structure to best fit your team's strengths, size, and working style.

Time Management and Expectations

Estimating the time required for a team of five students to complete the project involves considering various factors, such as individual skill levels, collaboration efficiency, and the complexity of the tasks. The specifications include the estimated time needed by the team to create the outcomes for each project phase.

These estimates assume that the students are working efficiently and have a basic understanding of Python and the project requirements. The actual time required may vary depending on prior experience, teamwork efficiency, and the complexity of the tasks.

The total time estimated for the project is 135 hours. I anticipate each team member will spend 2-3 hours per week in team meetings, plus 3-5 hours per week working independently on project outcomes.

Use of Generative AI in the Project

I expect you to use generative AI tools (e.g., ChatGPT, Claude, Google Gemini, GitHub Copilot, Perplexity) to support your project work. You may use these tools for activities such as brainstorming approaches, writing and refining code, debugging, documenting code, explaining concepts, evaluating alternative solutions, and improving your application.

You are also welcome to experiment with **vibe coding**, using AI tools to generate and refine portions of your project code through prompting. Regardless of how you use generative AI, your team remains responsible for the work you submit.

- **Explaining** – Each team member must be able to explain how the code the team submits works, including the major components of the application and how they work together.
- **Verifying** – Your team is responsible for reviewing, testing, debugging, and modifying AI-generated code as needed to verify that it works correctly and meets the project specifications.
- **Documenting** – Your team is responsible for documenting its use of generative AI in the Project Report. Identify the tools used, describe the types of prompts or questions used, explain how AI contributed to your work, and describe how your team evaluated or adapted its output.
- **Taking Responsibility** – Your team is responsible for the quality, accuracy, functionality, documentation, and completeness of the work you submit, regardless of how much or how little generative AI you use.

The intent is for you to explore the role of AI as a collaborative assistant while developing your coding skills. Your ability to explain, verify, adapt, and integrate AI-generated work is more important than the raw output of an AI tool.

Phase Outcomes

Each phase lists bullet points that detail high-level outcomes for the phase.

Additional Guidelines

- Focus on creating a user-friendly interface.
- Ensure the application handles errors gracefully and provides informative error messages.
- Document your code thoroughly to explain the purpose and function of the overall program, user-defined functions, complex code blocks, and variables.
- Consider edge cases and implement exception handling to manage unexpected events.
- Evaluate and test the compatibility of any external code modules with your application.

Phase 1: Project Proposal (Due August 30, 2026)

In this phase, you will plan the development of the Personal Finance Tracker application.

Requirements

- Develop a detailed project proposal outlining the application's features, design, and functionality.
- Describe some exceptions to handle and data validation to consider during the project.
- Identify any external libraries your team plans to use and explain why they are appropriate. If you do not plan to use external libraries, briefly explain why they are not needed.
- Create a timeline for development, specifying tasks and deadlines for each team member.

Generative AI: Use generative AI tools to brainstorm possible features, explore alternative designs, develop or refine your project plan, and identify potential risks, exceptions, and data validation issues to consider.

Suggested Challenge Levels

- Beginner: Basic feature description and simple design outline.
- Intermediate: Detailed feature description with technical specifications and preliminary mockups.

- Advanced: Comprehensive feature description, technical specifications, mockups, and a thorough risk assessment.

Suggested Team Roles and Responsibilities

Feel free to modify the suggested work breakdown structure to fit your team's strengths, size, and working style.

- Team Leader: Coordinate the proposal development and ensure all sections are complete.
- Developer(s): Contribute to the design and technical specifications.
- Tester/Documenter: Compile and format the proposal document.

Time Management

Plan for approximately 25 total team hours to complete Phase 1, distributed roughly as follows.

- Team Meetings: 5 hours
- Research and Planning: 10 hours
- Writing the Proposal: 10 hours

Phase Outcomes

- A comprehensive project proposal document, including all elements above.

Phase 2: Transaction Recording (Due September 13, 2026)

In this phase, you will implement the core functionality to record income and expenses.

Requirements

- Develop a Python application to input transactions (date, description, category, amount, type: income or expense).
- Store transactions in a list or a simple file-based database (e.g., CSV).
- Allow users to categorize transactions (e.g., food, rent, utilities, entertainment).
- Implement functions to calculate total income, expenses, and net savings.

Generative AI: Use generative AI tools to help write and refine code for transaction input, storage, categorization, and calculations; explore alternative approaches; and identify, explain, and debug errors.

Suggested Challenge Levels

- Beginner: Basic input functionality and simple storage in a list.
- Intermediate: Input validation and storage in a CSV file.
- Advanced: Detailed input validation, storage in a CSV file, and preliminary error handling.

Suggested Team Roles and Responsibilities

Feel free to modify the suggested work breakdown structure to fit your team's strengths, size, and working style.

- Team Leader: Coordinate tasks, ensure complete documentation, and integrate parts.
- Developer 1: Implement input functionality and user validation.
- Developer 2: Handle data storage and basic transaction recording.
- Developer 3: Focus on error handling and edge cases.
- Tester/Documenter: Test the application and document the code.

Time Management

Plan for approximately 35 total team hours to complete Phase 2, distributed roughly as follows.

- Team Meetings: 5 hours
- Coding: 15 hours
- Testing and Debugging: 10 hours
- Documentation: 5 hours

Phase Outcomes

- A Python application that records transactions and saves them to a file.
- Documentation explaining the purpose and functionality of the code.

Phase 3: Reporting and Exception Handling (Due September 27, 2026)

In this phase, you will add reporting features and data validation to your application.

Requirements

- Design at least two reports (e.g., expenses by category, spending by time, profit by time, etc.).
- Add robust data validation for user inputs.
- Build exception handling for some common, anticipated user issues.
- Create a command-line interface (CLI).

Generative AI: Use generative AI tools to help write and refine code for reports, data validation, exception handling, and the command-line interface; explore alternative approaches; and identify, explain, and debug errors.

Suggested Challenge Levels

- Beginner: Basic report generation with minimal validation and basic CLI.
- Intermediate: Enhanced reports with detailed validation and intermediate CLI.
- Advanced: Comprehensive reports, detailed validation, robust exception handling, and advanced CLI.

Suggested Team Roles and Responsibilities

Feel free to modify the suggested work breakdown structure to fit your team's strengths, size, and working style.

- Team Leader: Manage reporting features and ensure timely task completion.
- Developer 1: Create reports.
- Developer 2: Implement data validation.
- Developer 3: Build exception handling.
- Tester/Documenter: Test all new features, ensure integration with previous phases, and update documentation.

Time Management

Plan for approximately 35 total team hours to complete Phase 3, distributed roughly as follows.

- Team Meetings: 5 hours
- Coding: 15 hours
- Testing and Debugging: 10 hours
- Documentation: 5 hours

Phase Outcomes

- A Python application that generates reports and implements appropriate data validation.
- Documentation for the reporting and visualization components, including any error handling implemented.

Phase 4: User Interface and Final Integration (Due October 11, 2026)

In this phase, you will finalize the user interface and integrate all functionalities. A graphical user interface (GUI) is not required. Your team may continue using a command-line interface (CLI) or develop a graphical user interface (GUI), depending on the challenge level and design approach you choose.

Requirements

- Create at least one visualization for each report created for Phase 3 (e.g., bar charts, pie charts, etc.) using Turtle Graphics or another appropriate visualization tool.
- Integrate all functionalities (transaction recording, categorization, reporting, and visualization).
- Ensure the application is user-friendly and well-documented.

Generative AI: Use generative AI tools to help write and refine code for visualizations, the user interface, and application integration; explore alternative approaches; and identify, explain, and debug errors.

Suggested Challenge Levels

- Beginner: Refine the existing CLI, integrate all features, and create at least one visualization for each report using Turtle Graphics or another basic visualization approach.

- Intermediate: Develop a GUI with interactive elements using a library such as tkinter and create additional or enhanced visualizations.
- Advanced: Develop an enhanced GUI with advanced features and seamless integration, and use libraries such as Plotly or Matplotlib to create more sophisticated visualizations.

Suggested Team Roles and Responsibilities

Feel free to modify the suggested work breakdown structure to fit your team's strengths, size, and working style.

- Team Leader: Coordinate final development and integration, ensuring all components work together.
- Developer 1: Refine or extend the user interface.
- Developer 2: Develop and integrate visualizations.
- Developer 3: Integrate functionality from previous phases and address remaining technical issues.
- Tester/Documenter: Test the complete application, verify integration, and finalize comprehensive documentation.

Time Management

Plan for approximately 40 total team hours to complete Phase 4, distributed roughly as follows.

- Team Meetings: 5 hours
- Coding: 20 hours
- Testing and Debugging: 10 hours
- Documentation: 5 hours

Phase Outcomes

- A complete Personal Finance Tracker application with a user interface, ready for use.
- Comprehensive documentation covering the entire application, including user-defined functions, complex code blocks, and user-defined variables.

Project Management

Effective project management is crucial for the successful completion of your Python project. This section outlines essential practices in time management, code organization, and documentation to help you and your team work efficiently and produce high-quality results.

Internal Code Organization

Organize your code effectively to create maintainable and efficient software. Follow these principles to ensure clarity and functionality:

- Main Module: Include a "`main()`" function to control the overall program flow. This central function should coordinate the execution of different parts of your application, making it easier to manage and understand.
- Modularity: Break your code into self-contained modules and functions. Each module focuses on a specific task or functionality. Design functions to perform a single, well-defined task and make them reusable whenever possible.
- Focused Functions: Ensure that each function or module focuses on a specific task. This approach makes your code more understandable and easier to maintain.
- Independence: Design your modules and functions to be as independent as possible. Minimize interdependencies between different parts of your code to reduce complexity and increase the flexibility of your application.
- Consistent Naming Conventions: Use clear and consistent naming conventions for variables, functions, and classes to enhance code readability.
- File Organization and Directory Structure: Organize your project files into logical directories (e.g., '`src`' for source code, '`data`' for data files, '`/images`' for image files, and '`docs`' for documentation). See Figure 1 for an example directory structure.

```
/project-root
  /data
    -transactions.csv
    -categories.csv
  /docs
    -Phase-1-Project-Proposal.pdf
    -Phase-2-Project-Report.pdf
    -Phase-3-Project-Report.pdf
    -Phase-4-Project-Report.pdf
    -architecture_diagram.png
  /images
    -bar-chart.png
  /src
    -main.py
    -transactions.py
    -summaries.py
```

Figure 1 - Example Directory Structure

By following these principles of internal code organization, you will create a well-structured and maintainable project that is easier to debug, test, and extend.

Documenting Your Code

Proper documentation is crucial for the development and maintenance of high-quality software. It ensures that your code is understandable, maintainable, and easily reviewed by others. As you write your code, be diligent about documenting your work thoroughly and clearly. Add the program header shown in Figure 2 (with appropriate modifications) for each .py file.

```
# PROGRAM:       Name of the problem/program
# PURPOSE:       A brief description of what the program is supposed
                 to accomplish.
# INPUT:         Data that must be collected by the program in order
                 for it to run.
# PROCESS:       Actions performed on the data by the program to
                 generate appropriate outputs.
# OUTPUT:        Outputs generated by the program.
# HONOR CODE:    On my honor, as an Aggie, I have neither given nor
                 received unauthorized aid on this academic work.
```

Figure 2 - Example Program Header

In addition to the program header, add relevant inline comments throughout your code. This includes explaining complex logic, detailing the purpose of functions and variables, and noting any assumptions or decisions made during development. Following the documentation practices from the course coding guidelines will enhance the readability and quality of your code.

Deliverables

You will submit your project proposal as a single PDF file for the Phase 1 deliverable.

You will submit two files in Canvas for Phases 2-4:

1. Project Report
2. Project Files

This section details the contents of each file.

Project Report

For **Phases 2-4**, submit an updated Project Report documenting the current state of your application and the work completed during the phase. The Project Report should include the following:

- Project Title and Team Information: Provide the project title and list all team members, including their roles and contributions.
- Introduction and Purpose: Summarize the project's main objectives and the problems it aims to solve.
- Overview of Features: Describe the key features of your application, explaining what the program does and how it benefits the user.
- Main Program File: Identify the main program file to execute to run the application, with detailed instructions for setup or installation.
- File Descriptions: Explain the purpose of each file included in the submission, including code modules, data files, and any other relevant files, and how they fit into the overall project.
- User Instructions: Provide step-by-step instructions for how to use the application, including examples of input and expected output.
- Assumptions and Design Decisions: Document assumptions made during the project and explain the reasoning behind major design decisions.
- Challenges and Solutions: Discuss significant challenges encountered and how they were addressed, highlighting innovative solutions or approaches.
- Use of Generative AI: Describe how your team used generative AI in this phase. Include the types of prompts you tried, how AI shaped your approach, and how you verified and adapted its output. Be specific about the value it added or the challenges you encountered.
- Future Improvements: Suggest improvements or additional features that could enhance the application in future iterations.
- Honor Code Statement: Include the honor code statement, affirming that all work was completed while following academic integrity standards.

Ensure that the Project Report is well-organized, clear, and professional. This document will be a key reference for anyone reviewing or using your project, so take the time to make it thorough and informative. Save the Project Report as a PDF file and include it with your submission for Phases 2-4, updating and revising it as needed.

Project Files

Proper file management is crucial for maintaining an organized and efficient workflow. Throughout the project, you will create multiple files that need to be carefully managed

and documented. You might choose to organize and store your code on a platform like GitHub (or Texas A&M GitHub) for ease in managing versions and files.

For **Phases 2-4**, bundle all project files into a single ZIP file. This ensures that all components of the project are easily accessible and well organized. Your ZIP file for each phase should include, at a minimum, the following:

- All Code Modules: Include all .py files that comprise your project's codebase. Each code module should be well-documented with appropriate comments and headers as described in the documentation guidelines.
- Data Files (if any): If your project utilizes any data files (e.g., CSV files for storing transactions), ensure they are included in the ZIP file. These files should be properly formatted and relevant to the functionality of your application.
- Image Files (if any): If your project includes any visual elements (e.g., images used in the GUI or for visualization purposes), make sure to include these files. They should be in appropriate formats (e.g., PNG, JPEG) and properly referenced in your code.
- Documentation: Include the project proposal and the project report for each deliverable.

Organize your files into a clear directory structure within the ZIP file. For example:

- `'/data'` for data files
- `'/docs'` for documentation files
- `'/images'` for image files
- `'/src'` for source code files

Preparing the Deliverables for Phases 2-4

1. Create a ZIP File: Bundle all your project files, including code modules, data files, image files, and other supporting files, into a single ZIP file.
2. Organize Your Files: Ensure your files are organized into a logical directory structure within the ZIP file.
3. Name the ZIP File: Use a clear and descriptive name for the ZIP file, such as `'TeamName_ProjectPhaseX.zip.'`
4. Upload to Canvas: Submit the files through the designated assignment submission link on Canvas by the specified deadline.

What to Submit in Canvas

Phase 1 Submission

- Project Proposal (PDF): Upload a single PDF file containing your Project Proposal.

Subsequent Phases (Phases 2, 3, and 4) Submissions

- Project Report (PDF): Submit a standalone PDF file of the updated Project Report.
- Project Files (ZIP): Submit a ZIP file containing your Python source code and all supporting files.

Following these file management and submission guidelines will help ensure your project is organized, well-documented, and easy to review.

Grading Rubric

Table 1 shows the rubric for the project proposal phase. Error: Reference source not found shows the rubric for each coding phase.

The rubrics outline the criteria for assessment, ensuring consistent and fair evaluation across all phases. By adhering to these criteria, you can focus your efforts on meeting the project requirements effectively. The rubrics provide clear guidelines on how your work will be assessed, helping you achieve the highest standards in your project deliverables.

Table 1 - Rubric for Project Proposal Phase

Criteria	Excellent (100%)	Outstanding (80%)	Proficient (60%)	Developing (40%)	Emerging (20%)
Project Overview (500 points)	Complete and clear overview, well-defined objectives and scope.	Clear overview with mostly well-defined objectives and scope.	General overview with some objectives and scope defined.	Limited overview with vague objectives and scope.	Minimal overview with unclear objectives and scope.

Criteria	Excellent (100%)	Outstanding (80%)	Proficient (60%)	Developing (40%)	Emerging (20%)
Design and Features (1000 points)	Detailed and innovative design, all features clearly described.	Detailed design with most features clearly described.	General design with some features described.	Limited design with few features described.	Minimal design with unclear features.
Data Validation and Exceptions (1000 points)	Comprehensive description of data validation and exception handling strategies.	Detailed description of data validation and some exception handling strategies.	General description of data validation and minimal exception handling strategies.	Limited description of data validation and vague exception handling strategies.	Minimal description of data validation and unclear exception handling strategies.
External Libraries (500 points)	Clearly identifies and justifies any external libraries the team plans to use or clearly explains why external libraries are not needed.	Identifies any external libraries the team plans to use or explains why external libraries are not needed, with some justification.	Identifies potential external libraries or indicates that none are planned, with limited justification.	Provides limited information about potential external libraries and little justification for their use or exclusion.	Provides minimal or no information about potential external libraries or whether they will be used.
Timeline and Task Allocation (500 points)	Detailed timeline with specific tasks and deadlines for each team member.	Clear timeline with tasks and deadlines for most team members.	General timeline with some tasks and deadlines.	Limited timeline with vague tasks and deadlines.	Minimal timeline with unclear tasks and deadlines.

Criteria	Excellent (100%)	Outstanding (80%)	Proficient (60%)	Developing (40%)	Emerging (20%)
Generative AI Use (500 points)	Clearly documents how generative AI was used and thoroughly explains how the team evaluated, verified, adapted, or integrated its output.	Clearly documents how generative AI was used and explains how the team evaluated, verified, adapted, or integrated its output.	Documents how generative AI was used with some explanation of how the team evaluated, verified, adapted, or integrated its output.	Provides limited documentation of generative AI use and little explanation of how the team evaluated, verified, adapted, or integrated its output.	Provides minimal or no documentation of generative AI use or how the team evaluated, verified, adapted, or integrated its output.
Documentation and Formatting (500 points)	Well-organized, clear, and professional proposal document.	Mostly well-organized and clear proposal document with minor clarity or formatting issues.	Generally organized proposal document with some clarity or formatting issues.	Limited organization with substantial clarity or formatting issues.	Minimal organization with extensive clarity or formatting issues.
Writing Mechanics (500 points)	The submission has minimal (if any) grammar, punctuation, spelling, etc. errors.	The submission has a few grammar, punctuation, spelling, etc. errors.	The submission has several grammar, punctuation, spelling, etc. errors.	The submission has many grammar, punctuation, spelling, etc. errors.	The submission has extensive grammar, punctuation, spelling, etc. errors.

Table 2 – Rubric for Project Coding Phases

Criteria	Excellent (100%)	Outstanding (80%)	Proficient (60%)	Developing (40%)	Emerging (20%)
Inputs (1000 points)	Clearly labeled, appropriate data types, verified and validated, errors handled.	Most inputs labeled, appropriate data types, mostly verified and validated, most errors handled.	Some inputs labeled, appropriate data types, some verified and validated, some errors handled.	Few inputs labeled, appropriate data types, few verified and validated, few errors handled.	Minimal inputs labeled, appropriate data types, minimal verified and validated, minimal errors handled.
Process (1500 points)	Appropriate length/complexity, appropriate scope, well-designed logic.	Most methods/functions appropriate length/complexity, scope, logic.	Some methods/functions appropriate length/complexity, scope, logic.	Few methods/functions appropriate length/complexity, scope, logic.	Minimal methods/functions appropriate length/complexity, scope, logic.
Outputs (1000 points)	No/minimal errors, correct content, well-organized design.	Few errors, mostly correct content, mostly well-organized design.	Some errors, some correct content, some well-organized design.	Most errors, little correct content, little well-organized design.	Extensive errors, minimal correct content, minimal organization.
Program Execution (500 points)	Executes correctly, no syntax/runtime errors.	Launches, one or two runtime errors.	Launches, several runtime errors.	Does not execute due to syntax errors.	Does not execute due to several syntax errors.

Criteria	Excellent (100%)	Outstanding (80%)	Proficient (60%)	Developing (40%)	Emerging (20%)
Generative AI Use (500 points)	Clearly documents how generative AI was used and thoroughly explains how the team evaluated, verified, adapted, or integrated its output.	Clearly documents how generative AI was used and explains how the team evaluated, verified, adapted, or integrated its output.	Documents how generative AI was used with some explanation of how the team evaluated, verified, adapted, or integrated its output.	Provides limited documentation of generative AI use and little explanation of how the team evaluated, verified, adapted, or integrated its output.	Provides minimal or no documentation of generative AI use or how the team evaluated, verified, adapted, or integrated its output.
Documentation (500 points)	Thoroughly documents the code, adheres to all coding conventions, and clearly explains functionality.	Documents most of the code, adheres to most coding conventions, and clearly explains most functionality.	Documents some of the code, adheres to some coding conventions, and explains most major functionality.	Provides limited code documentation, adheres to few coding conventions, and partially explains functionality.	Provides minimal code documentation, adheres to few or no coding conventions, and provides minimal explanation of functionality.

Generative AI Disclosure

In keeping with my commitment to leverage advanced technology for enhanced efficiency and accuracy in my work, I use generative artificial intelligence tools to assist in drafting course materials.